

Class 1:-

input() Function

This function is used to read a variable's value from the user while running the program (Dynamic Input).

It always reads the value from the user as a STRING.

Syntax:

```
input(Prompt Message)
```

The message helps to tell the user what type of value they need to give as input.

Note: When we apply the + operator on a String --> Concatenation (Joining)

print() Function

This function is used to print a message along with some values in the output terminal.

This pre-defined function is created as:

```
print(*args, sep=' ', end='\n', file=None, flush=False)
```

Prints the values to a stream, or to sys.stdout by default.

- sep: string inserted between values, default a space.
- end: string appended after the last value, default a newline.
- file: a file-like object (stream); defaults to the current sys.stdout.
- flush: whether to forcibly flush the stream.

1. Comma Operator (,)

It is used to do the work of concatenation. It automatically adds a single space.

It can be used on all data types without any type conversion.

Example:

```
a = int(input())
b = int(input())
total = a+b
print(a,"+",b,"=",total)
```

2. Plus Operator (+)

It is used to do the work of concatenation.

Diff:

It will not add any extra space. If space is required then you have to type the code accordingly.

Works Only on Strings.

Example:

```
name = "Anu"
age = 28
# Expected Output: Hi this is Anu and i am 28yrs old
print("Hi this is " + name + " and i am " + age + "yrs old") --> TypeError
print("Hi this is " + name + " and i am " + str(age) + "yrs old")
```

3. F-String (Formatting String)

Prefix: F (or) f

Place Holder: {variableName}

Example:

```
language = "Java" # str
version = 23      # int
```

```
fees = 5750.25 # float
```

Expected Output: I am learning Java version 23. I paid a fees of Rs.5750.25

```
print(F"I am learning {language} version {version}. I paid a fees of Rs.{fees}")
```

Example:

```
name = input("Enter your name: ")
```

```
lang = input("Enter your favourite programming Language: ")
```

```
print(F"{name}'s favourite programming language is {lang}")
```

4. format() function

pre-defined string function.

String: Group of characters enclosed within quotations.

Characters can be --> alphabets (uppercase & lowercase), digits, special characters.

In Python: String can be created using: Single Quotes, Double Quotes, Triple Quotes.

```
a = 'Java' #str
```

```
b = "Java" #str
```

```
c = """Java""" #str
```

```
d = """"Java"""" #str
```

Place Holder: {}

Syntax: str.format(variable names separated with comma)

Order of the variable name is very important.

Example 1:

```
language = "Java" # str
```

```
version = 23 # int
```

```
fees = 5750.25 # float
```

Expected Output: I am learning Java version 23. I paid a fees of Rs.5750.25

```
print("I am learning {} version {}. I paid a fees of Rs.{}".format(language,version,fees))
```

Example 2:

a = 10

b = 20

total = "{} + {} = {}"

print(total.format(a,b,(a+b)))

Type Conversion: converting the value from one data type to another.

this task is performed with the help of pre-defined functions.

int(), float(), str(), complex(), bool(), list(), tuple(), set(), dict()

Nested Functions --> Writing one function inside of another.

Order of Execution - inner to outer

=====
=====
=====

CLASS 2:

Data Types

Data - information

Data Types --> refers to the type of information that i am going to be storing.

In C/C++/Java/C# Programming - create a variable:

```
dataType variableName = value;
```

Based on the data type mentioned - memory space will get allocated.

Note: 1 byte = 8 bits

1. Integer value --> Whole numbers (positive / negative)

types: byte, short, int, long

byte: 1 byte

short: 2 bytes

int: 4 bytes

long: 8 bytes

2. Real Numbers --> fractional (or) Decimal values

types: float, double

float: 4 bytes

double: 8 bytes

3. Character --> represents a single character enclosed within a single quotations

type: char

C/C++ --> 1 byte

Java --> 2 bytes

4. Boolean --> Used to store True (or) False

type: boolean / bool --> 1 bit

True = 1

False = 0

5. Strings --> group of characters enclosed within quotations.

Characters include: Alphabets (uppercase/lowercase), numbers/digits, special symbols !@#\$%^&*()_+*/:;.

Generally Programming Languages are classified into 2 categories

A. Statically Typed Programming Language

The variables must be declared along with the data type before making use of it in the program.

Uses Compilers

Programming Language Example: C, C++, Java, C#

Syntax: dataType variableName = value;

Example:

```
#include <stdio.h>

int main()
{
    int a,b,c; // Variable Declaration

    a = 10;
    b = 20;
    c = a+b;
    printf("%d",c);
    return 0;
}
```

B. Dynamically Typed Programming Language

No need to declare the variables, directly use them in the program.

Based on the value given to the variable, the data type is automatically decided by the interpreter.

Uses Interpreters

Programming Language Example: Python, Javascript, R

Syntax: varName = value

Example:

```
a = "Python "  
b = "Programming"  
c = a+b  
print(c);
```

A Computer can understand only 0's and 1's (Binary Language / Machine Language)

Translator Software:

1. Compiler --> scans the entire program and gives a consolidated list of all the errors i have done in the code.

Example:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    a = 10;
```

```
    b = 20;
```

```
    c = a+b;
```

```
    printf("%d",c)
```

```
    return 0;
```

```
}
```

Error List:

main.c: In function 'main':

main.c:5:5: error: 'a' undeclared (first use in this function)

```
5 |   a = 10;
```

```
  |   ^
```

main.c:5:5: note: each undeclared identifier is reported only once for each function it appears in

main.c:6:5: error: 'b' undeclared (first use in this function)

```
6 |   b = 20;
```

```
  |   ^
```

main.c:7:5: error: 'c' undeclared (first use in this function)

```

7 |   c = a+b;
  |   ^
main.c:8:19: error: expected ';' before 'return'
8 |   printf("%d",c)
  |               ^
  |               ;
9 |   return 0;
  |   ~~~~~

```

2. Interpreter: Line by Line execution of the Code.

Whenever it encounters an error, the program will stop the execution and print that error for us.

Whatever line of code is there below that error is ignored.

Example:

```
a = 25
```

```
b = 85
```

```
print(c)
```

```
print(d)
```

Error:

Traceback (most recent call last):

```
print(c)
```

NameError: name 'c' is not defined

Python - Data Types

All the data types in Python are in-built. Based on the value given for a variable, the data type is automatically decided.

14 data types.

A. Fundamental Data Types(5): int, float, str, bool, complex

Note: Python do not support: double, char, long, byte, short

B. Collection Types (7): Inside a variable storing multiple values.

list, tuple, set, dict, frozenset, bytes, bytearray

C. Others(2): range, None

Example:

Fundamental Types

a = 15 #int

b = 56.36 #float

c = "Welcome to Python" #str

d = True #bool

e = 10 + 5j #complex

print(F"Value = {a} and Data Type = {type(a)}")

print(F"Value = {b} and Data Type = {type(b)}")

print(F"Value = {c} and Data Type = {type(c)}")

print(F"Value = {d} and Data Type = {type(d)}")

print(F"Value = {e} and Data Type = {type(e)}")

14 data type can further be classified into:

A. Mutable Data Types - data can be modified (insert, delete, update)

list, set, dict, bytearray.

B. Immutable Data Types - data that cannot be modified;

Once the data is created it cannot be changed (constant/fixed)

int, float, str, bool, tuple, frozenset, bytes, range, None

Built-in Functions

1. `type(value)`: returns the data type of the value specified within it.

```
=====
=====
=====
```

CLASS 3:

Built-in Functions

1. `type(value)`: returns the data type of the value specified within it.

Example:

```
print(type(10))
```

```
print(type(52+6j))
```

```
a = "Python"
```

```
print(type(a))
```

2. `id(variableName)`: return the memory address/location where the data is being stored

Example:

```
# \t --> Horizontal Tabspace
```

```
# \n --> Newline
```

```
a = 456
```

```
b = 789.63
```

```
c = "Welcome"
print("Value \t Data type \t Address")
print(F"{a}\t{type(a)}\t{id(a)}")
print(F"{b}\t{type(b)}\t{id(b)}")
print(F"{c}\t{type(c)}\t{id(c)}")
```

In C Programming: Created 3 integer variables with the same value.

Each and every variable created has its own memory space, inside which the value is stored.

```
#include <stdio.h>
int main()
{
    int a,b,c;
    a = 10;
    b = 10;
    c = 10;
    printf("a=%d \t address=%u", a,&a);
    printf("\nb=%d \t address=%u", b,&b);
    printf("\nc=%d \t address=%u", c,&c);
    return 0;
}
```

Output:

```
a=10    address=3305905788
b=10    address=3305905784
c=10    address=3305905780
```

In Python - Memory is allocated differently for Mutable & Immutable Data Types

For Immutable Data Type: Memory space gets allocated based on the VALUE.

Example 1:

Integers - Immutable

a = 10

b = 10

c = 10

print("Value \t Address")

print(F"{a}\t{id(a)}")

print(F"{b}\t{id(b)}")

print(F"{c}\t{id(c)}")

print()

Strings - Immutable

s1 = "C++"

s2 = "C++"

s3 = "C++"

print("Value \t Address")

print(F"{s1}\t{id(s1)}")

print(F"{s2}\t{id(s2)}")

print(F"{s3}\t{id(s3)}")

print()

Example 2

Integers - Immutable

a = 10

b = 10

c = 10

print("Value \t Address")

print(F"{a}\t{id(a)}")

```
print(F"{b}\t{id(b)}")
print(F"{c}\t{id(c)}")
print()
```

Changing the value of 'c'

```
c = 20
d = 10 # A new variable 'd' and making it point to 10
print("Value \t Address")
print(F"{c}\t{id(c)}")
print(F"{d}\t{id(d)}")
```

For Mutable Data types: For every variable created, separate address will get created.

Example 1:

List - Mutable

```
L1 = [10,20,30]
L2 = [10,20,30]
print(L1, type(L1), id(L1))
print(L2, type(L2), id(L2))
```

Inserting a new data to the end of the list: [10,20,30,50]

```
L1.append(50)
```

Modifying the data: Index 2 currently contains 30 ; Update that value with 'Java' -
[10,20,'Java',50]

```
L1[2] = 'Java'
```

Deleting the 1st element in the list: [20,'Java',50]

```
L1.pop(0)
print(L1, type(L1), id(L1))
```

VARIABLE

A name that is used to store a value temporarily in the memory.

Syntax:

```
varName = value
```

Example:

```
# Creating a variable & assigning the value for it
```

```
a = 789
```

```
print("a =", a)
```

```
# Creating multiple variables and assigning values - in a single line
```

```
b, c = 45.6, 'Java'
```

```
print("b =", b)
```

```
print("c =", c)
```

Example:

```
# Program: Swapping the values of 2 variables - Using a temp variable
```

```
# All programming languages: this logic will work
```

```
a = 25
```

```
b = 30
```

```
print(f"Before Swapping: a={a} and b={b}")
```

```
temp = a
```

```
a = b
```

```
b = temp
```

```
print(f"After Swapping: a={a} and b={b}")
```

```
print()
```

```
# Specially in Python - Not additional variable: Memory efficient
```

```
a = 'Java'
b = 25
print(F"Before Swapping: a={a} and b={b}")
a,b = b,a
print(F"After Swapping: a={a} and b={b}")
print()
```

Formula to swap without a temp variable - works only on integers

```
a = a+b
b = a-b
a = a-b
```

KEYWORDS

Every programming language has a set of pre-defined words known as Keywords (or) Reserved Words.

They are the building blocks of a program.

Every keyword has a special meaning, programmers are not allowed to modify it.

Python consist of 35 keywords

Printing All the Keywords in Python - Code

Module: keyword; inside this module kwlist is present.

```
import keyword
```

kwlist - it is a pre-defined variable which contains all the keywords

```
print(keyword.kwlist)
```

len() - number of elements present within the list

```
print("Number of Keywords = ", len(keyword.kwlist))
```

Output:

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue',  
'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is',  
'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']
```

Number of Keywords = 35

Note: All the keywords except: False, True, None, are in lowercase .

Number representation of Boolean values: True = 1, False = 0

```
=====
```

```
=====
```

```
=====
```

class 4:

Computational Thinking

Computational Thinking (CT) is a problem-solving process that involves formulating problems in a way that a

computer can effectively carry out.

i.e using computer-science concepts to solve problems, design systems, and understand human behaviour more efficiently.

The Four Pillars of Computational Thinking

1. Decomposition

Breaking a complex problem into smaller, manageable parts

Say for Example: Breaking "build a game" into rules, graphics, scoring, controls

2. Pattern Recognition

Finding similarities or trends among problems

Example: Noticing that sorting a list of names uses the same logic as sorting numbers

3. Abstraction

Focusing on important information only, ignoring irrelevant detail

Example: A car's steering wheel abstracts away the mechanics of the engine

4. Algorithm Design

Developing a step-by-step solution to the problem

Example: Writing the exact steps to sort a list

Problem Solving Methodology

1. Understand the Problem

What is being asked? What are the inputs? What is the expected output?

Ask: "Can I restate this problem in my own words?"

2. Analyze the Problem

Identify known, unknown, and constraints

Identify inputs and outputs clearly

3. Design a Solution (Algorithm)

Break the problem into steps (decomposition)

Represent the solution using an algorithm, flowchart, or pseudocode

4. Implement the Solution

Translate the algorithm into actual program code (Python)

5. Test and Debug

Run the program with different test cases (normal, boundary, and invalid inputs)

Fix errors (debugging)

6. Document and Maintain

Add comments, write documentation

Update the program as requirements change

Example: "Find the largest of three numbers"

1. Understand: Input: 3 numbers → Output: the largest number
 2. Analyze: Inputs: a, b, c (numeric); Output: max value
 3. Design: Compare a with b, then compare the winner with c
 4. Implement: Write Python code
 5. Test: Try (5,2,9), (1,1,1), (-3,-8,-1)
-
-

Algorithms

An algorithm is a finite set of well-defined, unambiguous instructions to solve a problem or perform a task, given in a logical sequence.

Characteristics of a Good Algorithm:

- | | |
|--------------------------|---|
| 1. Finiteness | Must terminate after a finite number of steps |
| 2. Definiteness | Each step must be clear and unambiguous |
| 3. Input | Zero or more well-defined inputs |
| 4. Output | At least one well-defined output |
| 5. Effectiveness | Each step must be simple enough to be carried out |
| 6. Language Independence | Should be implementable in any programming language |

Algorithm — Find the Largest of Three Numbers

Step 1: Start

Step 2: Read three numbers a, b, c

Step 3: If a > b then

 if a > c then

 max = a

 else

 max = c

else

 if b > c then

 max = b

 else

 max = c

Step 4: Print max

Step 5: Stop

Algorithm — Check if a Number is Even or Odd

Step 1: Start

Step 2: Read number n

Step 3: If $n \% 2 == 0$ then

 Print "Even"

else

 Print "Odd"

Step 4: Stop

Flowcharts

A flowchart is a diagrammatic (pictorial) representation of an algorithm using standard symbols

connected by arrows to show the flow of control/logic.

Standard Flowchart Symbols

Symbol	Name	Purpose
Oval / Rounded Rectangle flowchart	Terminal	Start / End of the
Parallelogram (Read/Print)	Input/Output	Data input or output
Rectangle processing step (assignment, calculation)	Process	Computation or
Diamond (Yes/No, True/False)	Decision	Conditional branching
Arrow of execution	Flow line	Shows direction/order
Circle parts, especially across pages	Connector	Connects flowchart
Rectangle with double side lines	Predefined process	Represents a subroutine or function call

Rules for Drawing Flowcharts

- Every flowchart must have exactly one Start and at least one End
- Flow generally goes top to bottom / left to right
- Use arrows to clearly indicate the direction of flow
- Decision boxes must have exactly two exits (Yes/No or True/False)
- Keep it simple — avoid crossing lines where possible

----- Pseudocode

Pseudocode is an informal, English-like way of representing an algorithm using structured but non-syntactic language.

It bridges the gap between an algorithm (plain steps) and actual code (Python syntax).

Common Pseudocode Keywords

Keyword	Meaning
BEGIN / END	Start/end of program or block
INPUT / READ	Take input from user
OUTPUT / PRINT	Display output
IF / ELSE / ENDIF	Conditional branching
WHILE / FOR / ENDLOOP	Looping/iteration
SET / =	Assignment

Example — Largest of Two Numbers

```
BEGIN
  READ A, B
  IF A > B THEN
    PRINT "A is largest"
  ELSE
    PRINT "B is largest"
  ENDIF
END
```

Example — Sum of First N Natural Numbers

```
BEGIN
  READ N
  SET sum = 0
  SET i = 1
  WHILE i <= N DO
```

```
sum = sum + i
i = i + 1
ENDWHILE
PRINT sum
END
```

TASK: Algorithm vs. Flowchart vs. Pseudocode

What is Software?

Software is a collection of programs, data, and instructions that tell a computer how to perform a task.

It is the non-physical component of a computer system (as opposed to hardware, the physical parts).

Types of Software

A. System Software Manages and controls computer hardware so that application software can perform a task.

B. Application Software Designed to help the user perform specific tasks.

C. Programming Software (Development Tools) Used by developers to create other software.

=====

=====

=====

class 5:

Data Types

14 types: int, float, str, bool, complex, list, set, tuple, dict, frozenset, bytes, bytearray, None, range

A. Mutable Data Types: data can be modified after creation.

Types: list, set, dict, bytearray

B. Immutable Data Types: data that cannot be modified after creation.

(Fixed/Constant)

Types: int, float, str, bool, complex, tuple, frozenset, bytes, None, range

Keywords

Also known as Reserved Words / Built-in Words

They are the building blocks of a program.

Python: 35 keywords

Code:

```
# Importing the pre-defined Module
```

```
# Inside this Module - kwlist variable is present .
```

```
# kwlist contains all the keywords that are present in Python
```

```
import keyword
```

```
print(keyword.kwlist)
```

```
print(len(keyword.kwlist))
```

Variables

A variable is a name inside which the value is stored temporarily in the memory.

Syntax:

```
varName = value
```

F-String (Formatted Strings)

- * It is used for printing messages in a particular format
- * Prefix: F (or) f
- * Placeholder for printing the variable's value --> {varName}

Example 1:

```
# Creating Variables & Assigning Values
```

```
# = is the Assignment operator
```

```
# Value on the RHS will be assigned to the variable on the LHS
```

```
a = "Anushya"
```

```
b = "Python"
```

```
c = "3.14"
```

```
d = 9381817369
```

```
e = "anushya.trainer@gmail.com"
```

```
# Printing the Variable's value
```

```
print(a)
```

```
print(b)
```

```
print(c)
```

```
print(d)
```

```
print(e)
```

Example 2:

Creating Variables & Assigning Values

= is the Assignment operator

Value on the RHS will be assigned to the variable on the LHS

```
a = "Anushya"
```

```
b = "Python"
```

```
c = "3.14"
```

```
d = 9381817369
```

```
e = "anushya.trainer@gmail.com"
```

Print --> Message + Variable's value

```
print(f"My name is {a}")
```

```
print(f"I am leaning {b}, version {c}")
```

```
print(f"My WhatsApp Number is {d}")
```

```
print(f"You can contact me at {e}")
```

Expected Output:

My name is Anushya.

I am learning Python, version 3.14.

My WhatsApp Number is 9381817369

You can contact me at anushya.trainer@gmail.com

Example 3:

```
print("Welcome to Python Programming Language.\nAll the Best!\tLearn it Well.....")
```

Output:

Welcome to Python Programming Language.

All the Best! Learn it Well.....

Example 4: Create multiple variables and assign values to them in a single line of code.

```
name = "Anu"
```

```
rno = 1255
cgpa = 8.7
print(F"Name={name}, Roll Number={rno}, CGPA={cgpa}")
print()
name, rno, cgpa = "Srikant", 150, 9.3
print(F"Name={name}, Roll Number={rno}, CGPA={cgpa}")
```

Output:

```
Name=Anu, Roll Number=1255, CGPA=8.7
Name=Srikant, Roll Number=150, CGPA=9.3
```

Example 5: How to delete a variable --> del : Once deleted, you cannot use that variable again.

(Removing the variable from memory)

If used - Error: NameError

Syntax: del VariableName

```
language = "Java"
print(F"My favourite programming language is {language}")
del language
print(F"My favourite programming language is {language}") # ERROR
```

Example 6: Interchanging variable's value (Swapping)

Input from User and storing it inside the variable

```
a = input("Enter the 1st Value: ") # 78
```

```
b = input("Enter the 2nd Value: ") # 96
```

Printing the a and b values before Swapping

```
print(F"Before Swapping: a={a} and b={b}") # a=78 and b=96
```

Logic : Python's Special Way

a , b = b , a # --> a,b = 96,78

Printing the a and b values after Swapping

print(F"After Swapping: a={a} and b={b}") # a=96 and b=78

Example 7: # What will be the Output?

x,y = 7,5 # x=7 and y=5

x, y, x = x+10, y+5, x+20

x, y, x = 7+10, 5+5, 7+20 --> x=17, y=10, x=27

print(x) # 27

print(y) # 10

Example 7: Number System

1. Decimal --> Base 10: 0 to 9 : Default Number System of Python

2. Binary --> Base 2: 0 and 1: prefix=0b

3. Octal --> Base 8: 0 to 7: prefix=0o

4. Hexadecimal --> Base 16: 0 to 9 and A to F : prefix=0x

Example:

a = 10000 #Decimal

b = 0b10101 #Binary

c = 0o4567 #Octal

d = 0xAF #Hexadecimal

Printing the values

```
print(F"a = {a}")
print(F"b = {b}")
print(F"c = {c}")
print(F"d = {d}")
```

Module --> It is a Python File which contains Functions/Variables

Pre-defined Modules Examples: keyword, random, date, os, sys, itertools, strings, collection.....etc

print():

Messages / values that needs to be printed in the output window/terminal - we use the print() function.

len():

returns an integer value that represents the total number of element present within the collection.

Escape Sequence:

\n : Newline --> Message will get printed in a Newline.

\t : Horizontal Tab Space

input("Prompt Message"):

Use: To read a variable's value during the execution of the program (Runtime), we use

input() function.

input() function always takes the value as a STRING.

You have to use the concept of Type Conversion to convert the String into other types.

Type Conversion means converting the data from one data type to another.

In Python it is done with the help of pre-defined Functions:

int(), float(), str(), complex(), bool(), list(), set(), tuple(), dict(), frozenset().....etc

=====

class 6:

Pre-defined Functions - Type Conversion

1. bin() - Function is used for converting a number into a Binary value.

prefix: 0b

Example:

```
a = int(input("Enter a Number: "))
```

```
result = bin(a) # Convert the 'a' values into a Binary Number
```

```
print(F"{a} equivalent Binary value = {result}")
```

2. oct() - Function is used for converting a number into its Octal value.

prefix: 0o

Example:

```
a = int(input("Enter a Number: "))
```

```
result = oct(a) # Convert the 'a' values into a Octal Number
print(F"{a} equivalent Octal value = {result}")
```

3. hex() - Function is used for converting a number into its hexadecimal value.

prefix: 0x

Example:

```
a = int(input("Enter a Number: "))
result = hex(a) # Convert the 'a' values into a Hexadecimal Number
print(F"{a} equivalent Hexadecimal value = {result}")
```

ASCII --> American Standard Code for Information Interchange

Number representation of the Keyboard characters (English Language)

Computers understand only Binary language (Machine Language) --> 0's and 1's

Alphabets:

Uppercase: A to Z --> 65 to 90

Lowercase: a to z --> 97 to 122

Digits: 0 to 9 --> 48 to 57

UNICODE --> International Format - support languages

4. ord(): You will give a character and the function will return the ASCII number of the character.

Example:

```
print(ord('A')) #65
```

```
print(ord('f')) #102
```

```
print(ord('5')) #53
```

```
print(ord('#')) #35
```

```
print(ord(',')) #44
```

```
print(ord(' ')) #32
```

5. chr(): You will give ASCII number and the function will return the ASCII character of that number.

Example:

```
print(chr(105)) # i
```

```
print(chr(92)) # \
```

```
print(chr(32)) # ' '
```

```
print(chr(48)) # 0
```

STRING FUNCTION: str.FunctionName()

6. lower(): It is used to convert the String into Lowercase alphabets

7. upper(): It is used to convert the String into Uppercase alphabets

Example:

```
s = input("Enter a value: ")
```

```
r1 = s.upper() # Converting the String value into uppercase
```

```
print(F"String = {r1}")
```

```
r2 = s.lower() # Converting the String value into lowercase
```

```
print(F"String = {r2}")
```

Difference between Function & Methods

OOPs (Object Oriented Programming) --> C++, Java, Python, C#

In OOPs - Methods

When we write a code without OOPs --> Functions

OPERATORS

1. Assignment Operator: =

It is used to assign a value that is on the RHS to the variable on the LHS

Example:

a = 45

b = "Java"

c = a

d = 78

e = a + d

2. Arithmetic Operators - (7 operators) Mathematical Operations

Addition: +

Subtraction: -

Multiplication: *

Division: / (Always gives the result as a FLOAT)

Floor Division: //(Always gives the result as an INTEGER)

Modulo (Reminder): %

Exponent: ** (Power of)

Note:

True is represented by the number 1

False is represented by the number 0

3. Arithmetic Assignment Operators (Short hand Operators)

+=, -=, *=, /=, //=, %=, **=

In Python we do not have: ++(Increment by 1) and -- (decrement by 1)

Example:

a = 20

```
a+=50 # a = a+50
```

```
print(a) # 70
```

```
x = 100
```

```
x*=500 # x= x*500
```

```
print(x) # 50000
```

4. Relational Operators - Comparison Operators

It is used to compare values.

Result: Boolean value (True (or) False)

Greater than : >

Lesser than : <

Greater than or equal to: >=

Lesser than or equal to : <=

equal to : ==

Not equal to : !=

Example:

```
print(10>10)
```

```
print(10>100)
```

```
print(10<20)
```

```
print(10>=10)
```

```
print(10==10)
```

```
print(10!=10)
```

5. Logical Operators

6. Bitwise Operators

7. Membership Operators

8. Identity Operators

9. Conditional Operator

```
=====
=====
=====
```

class 7:

Number representation of the Boolean values: True=1, False=0

5. Logical Operators

When we want to combine multiple conditions into a single condition - we use Logical operators

In Python - 3 types: and, or, not

Result of the operation is a boolean value.

Note:

In C / C++ / Java: &&(AND) , || (OR), ! (NOT)

In Python, we do not use Symbols

and Operator

When all of the combining conditions is True, then the result is True.

Condition 1	Condition 2	Result
True	True	True
True	False	False
False	True	False
False	False	False

or Operator

If anyone of the combining condition is True, then the result is True.

Condition 1	Condition 2	Result
True	True	True
True	False	True
False	True	True
False	False	False

not Operator (inverse)

True --> False

False --> True

Example:

a = 20

b = 10

c = 30

AND

print((a>b and a>c)) # (20>10 and 20>30) --> True and False = False

print((a>b and c>a)) # (20>10 and 30>20) --> True and True = True

OR

print((a==10 or b==10)) # (20==10 or 10==10) --> False or True = True

print((c>a or b>c)) # (30>20 or 10>30) --> True or False = True

NOT

print(not(a==10)) #True

Example: and Operator

Correct Data

userName = 'admin'

password = 'admin@123'

User Input

uname = input("Enter Your User Name: ")

pwd = input("Enter your Password: ")

Validate any condition: if-else

if will work when the condition is True

else will work when the condition is False

if ((userName==uname) and (password==pwd)):

print("Login Successful!")

else:

print("XXX Login failed XXXX")

Example: or Operator

User Input

uname = input("Enter Your User Name: ")

pwd = input("Enter your Password: ")

The User Name must be accepted in anycase

if ((uname=='admin') or (uname=='ADMIN')) and (pwd=='admin@123'):

print("Login Successful!")

else:

print("XXX Login failed XXXX")

6. Bitwise Operators

These operators work on Binary values.

You can apply these operators ONLY on INTEGERS (Whole number - positive / negative)

- a. & (AND)
- b. | (OR)
- c. ^ (XOR)
- d. ~ (2's Complement)
- e. << (Left Shift)
- f. >> (Right Shift)

A. Bitwise AND (&)

If both the bits are 1 - result is 1; any other combination result is 0

Example: $45 \& 25 = 9$

45: 101101

25: 011001

& : 001001 --> 9

Example: $78 \& 24 = 8$

78: 1001110

24: 0011000

& : 0001000 --> 8

B. Bitwise OR (|)

If both the bits are 0 - result is 0; any other combination result is 1

Example: $45 | 25 = 61$

45: 101101

25: 011001

| : 111101 --> 61

Example: $78 | 24 = 94$

78: 1001110

24: 0011000

| : 1011110 --> 94

C. Bitwise XOR (^) - exclusive

If both the bits are same - result is 0; otherwise result is 1

Example: $45 \wedge 25 = 52$

45: 101101

25: 011001

^ : 110100 --> 52

Example: $78 \wedge 24 = 86$

78: 1001110

24: 0011000

^ : 1010110 --> 86

D. Compliment Operator ~

1's Compliment --> Flip the bits (0-->1 and 1-->0)

2's Compliment --> Find 1's compliment + 1

SHORT CUT:

Positive Number: Result will be in Negative value; GivenNumber+1

~10

~45

~1500

~100

Negative Number: Result will be in Positive value; GivenNumber-1

~-85

~-563

~-1000

E. Left Shift

Shifting of bits --> remove bits from the LHS and add bits to the RHS

Syntax: $X \ll n$

X = The Number

n = Number of bits to shift

Binary Number --> base=2

SHORT CUT: Multiplication Method --> $X * 2^n$

Example:

$45 \ll 2 \rightarrow 45 \times 2^2 \rightarrow 45 \times 4 = 180$

$78 \ll 3 \rightarrow 78 \times 2^3 \rightarrow 78 \times 8 = 624$

$150 \ll 6 \rightarrow 150 \times 2^6 \rightarrow 150 \times 64 = 9600$

F. Right Shift

Shifting of bits --> remove bits from the RHS and add bits to the LHS

Syntax: $X \gg n$

X = The Number

n = Number of bits to shift

Binary Number --> base=2

SHORT CUT: Division Method --> $X/2^{**n}$

Example:

$$45 \gg 3 \rightarrow 45 // 2^{*3} \rightarrow 45 // 8 = 5$$
